CISC 5460 MongoDB Assignment

Part 1 – Data Setup

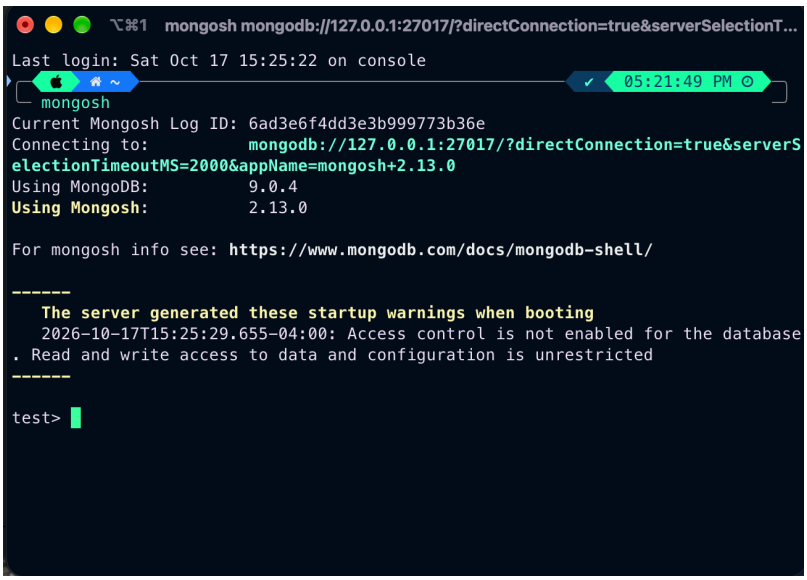


Figure 1: Terminal output shows a local instance connection using the mongosh command

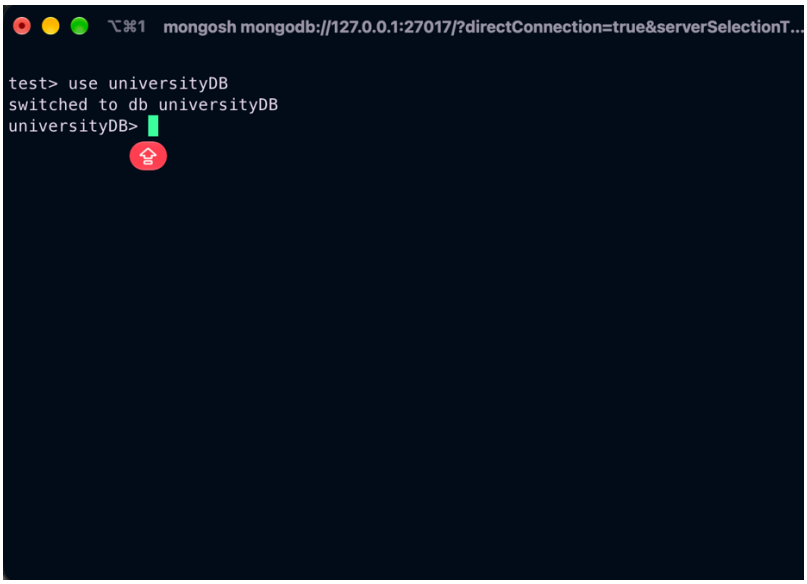


Figure 2: Terminal output shows the university database is switched to using the use command

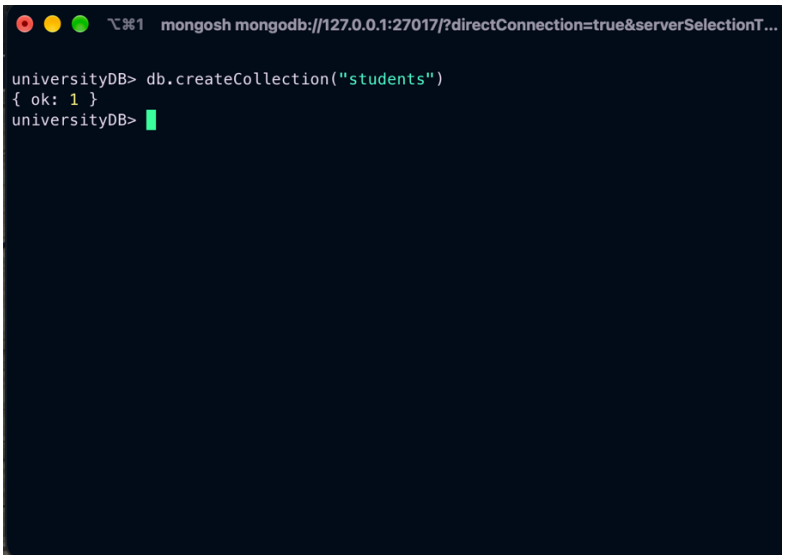


Figure 3: Terminal output shows a new collection in the university database called students is created

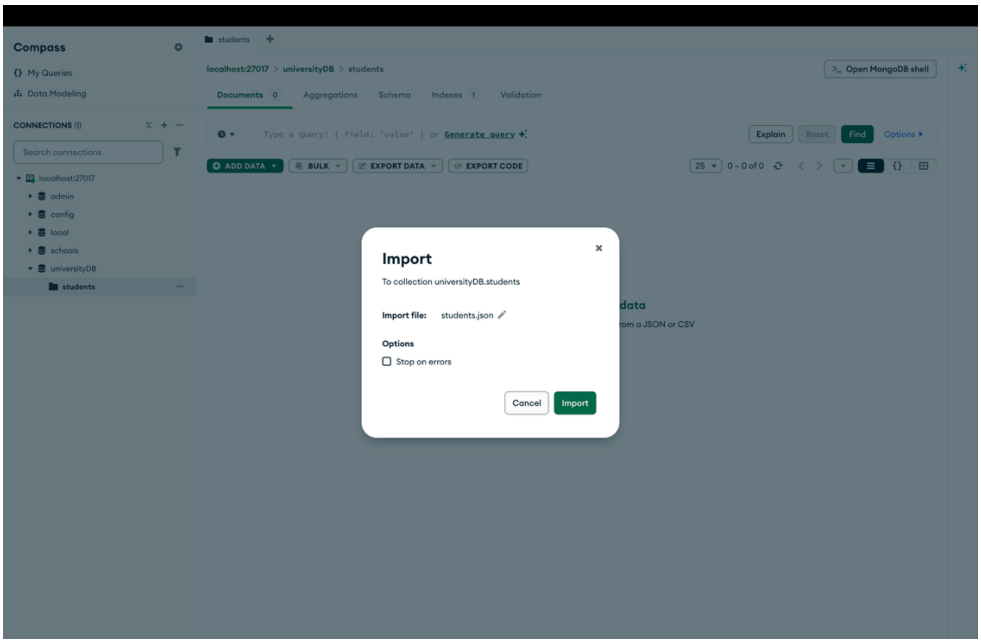


Figure 4: MongoDB compass was used to import the students' data

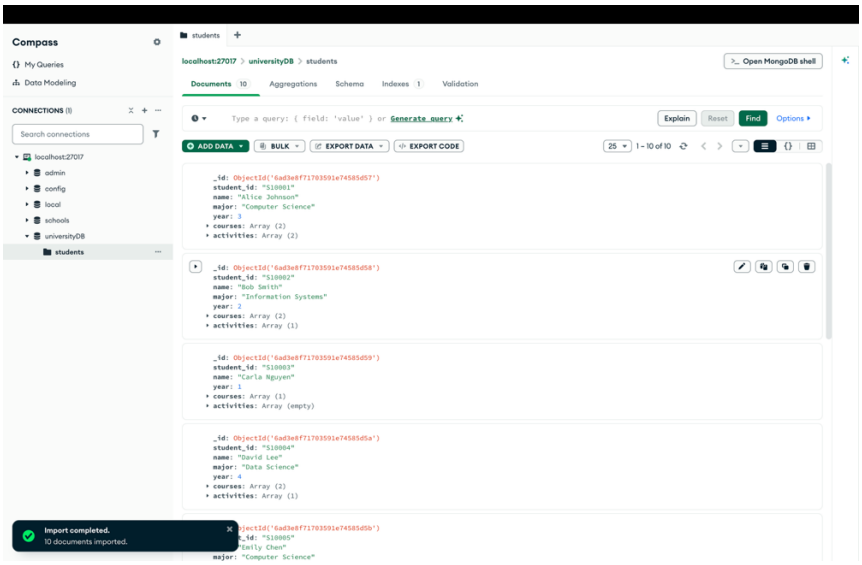


Figure 5: Screenshot shows the student documents in the collection

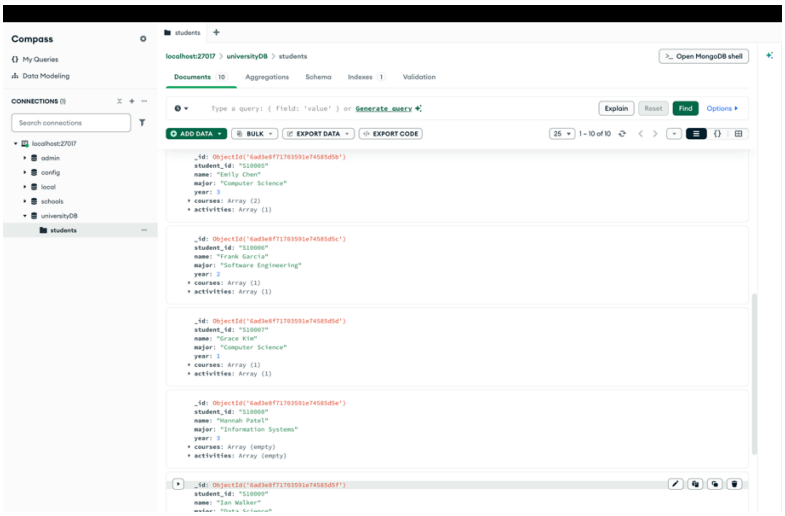


Figure 6: Screenshot shows the student documents in the collection

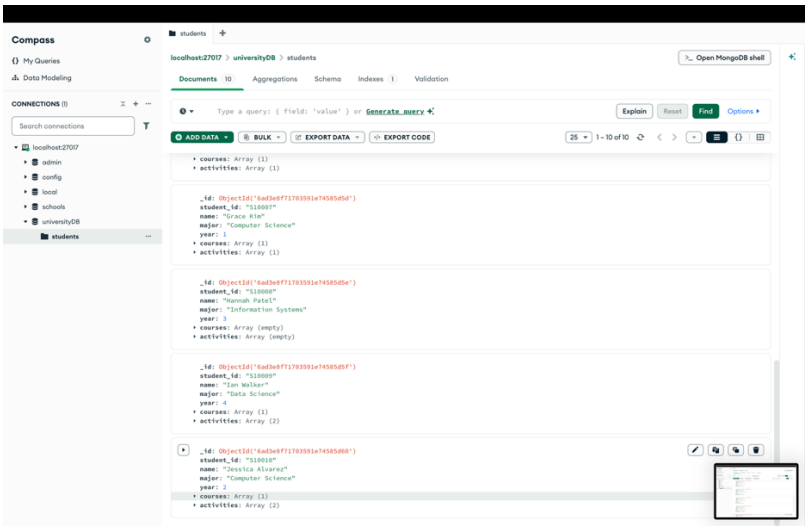


Figure 7: Screenshot shows the student documents in the collection

Part 2 – Querying

1. Retrieve all students majoring in "Computer Science

```
universityDB> db.students.find({major: "Computer Science"})
[
  {
    _id: ObjectId('6ad3e8f71703591e74585d57'),
    student_id: 'S10001',
    name: 'Alice Johnson',
    major: 'Computer Science',
    year: 3,
    courses: [
      { course_id: 'CS501', name: 'Advanced Databases', grade: 'A' },
      { course_id: 'CS502', name: 'Data Mining', grade: 'B+' }
    ],
    activities: [ 'AI Club', 'Robotics Club' ]
  },
  {
    _id: ObjectId('6ad3e8f71703591e74585d5b'),
    student_id: 'S10005',
    name: 'Emily Chen',
    major: 'Computer Science',
    year: 3,
    courses: [
      { course_id: 'CS502', name: 'Data Mining', grade: 'A-' },
      { course_id: 'CS503', name: 'Cloud Databases', grade: 'B' }
    ],
    activities: [ 'AI Club' ]
  },
  {
    _id: ObjectId('6ad3e8f71703591e74585d5d'),
    student_id: 'S10007',
    name: 'Grace Kim',
    major: 'Computer Science',
    year: 1,
    courses: [ { course_id: 'CS501', name: 'Advanced Databases', grade: 'A' } ],
    activities: [ 'Robotics Club' ]
  },
  {
    _id: ObjectId('6ad3e8f71703591e74585d60'),
    student_id: 'S10010',
    name: 'Jessica Alvarez',
    major: 'Computer Science',
    year: 2,
    courses: [ { course_id: 'CS503', name: 'Cloud Databases', grade: 'B' } ],
    activities: [ 'Software Club', 'AI Club' ]
  }
]
universityDB>
```

2. Retrieve all students who have taken the course CS502.



3. Retrieve the names of students who have a grade of A in any course

```
universityDB> db.students.find({"courses.grade" : "A"}, {_id: false, name: true})
[
  { name: 'Alice Johnson' },
  { name: 'David Lee' },
  { name: 'Grace Kim' }
]
universityDB>
```

4. Find students with no major field.

```
universityDB> db.students.find({major : {$exists: false}})
[
  {
    _id: ObjectId('6ad3e8f71703591e74585d59'),
    student_id: 'S10003',
    name: 'Carla Nguyen',
    year: 1,
    courses: [ { course_id: 'CS501', name: 'Advanced Databases', grade: 'B+' } ],
    activities: []
  }
]
universityDB>
```

5. Find students whose activities include "AI Club



Part 3 – Indexes

1. Create an index on the major field.

```
universityDB> db.students.createIndex({major: 1})
major_1
universityDB>
```

2. Create an index on courses.course_id.

```
universityDB> db.students.createIndex({"courses.course_id": 1})
courses.course_id_1
universityDB>
```

3. Compare query performance with and without the indexes for query #2 above.



Figure 8: Prior to setting an index, the query to find the students that took CS502 had a total of 10 documents examined.



Figure 9: After setting an index to the courses.course_id, a total number of two documents were examined as opposed to 10 prior to setting an index.

Part 4 - Aggregation Pipeline

1. Calculate the average grade for each course (map grades to numeric values for averaging: A=4, A-=3.7, B+=3.3, etc.).

```
universityDB> db.students.aggregate([
|   { $unwind: "$courses" },
|   {
|     $group: {
|       _id: {
|         course_id: "$courses.course_id",
|         course_name: "$courses.name"
|       },
|       average_grade: {
|         $avg: {
|           $switch: {
|             branches: [
|               { case: { $eq: ["$courses.grade", "A"] }, then: 4.0 },
|               { case: { $eq: ["$courses.grade", "A-"] }, then: 3.7 },
|               { case: { $eq: ["$courses.grade", "B+"] }, then: 3.3 },
|               { case: { $eq: ["$courses.grade", "B"] }, then: 3.0 },
|               { case: { $eq: ["$courses.grade", "B-"] }, then: 2.7 },
|               { case: { $eq: ["$courses.grade", "C+"] }, then: 2.3 },
|               { case: { $eq: ["$courses.grade", "C"] }, then: 2.0 },
|               { case: { $eq: ["$courses.grade", "C-"] }, then: 1.7 },
|               { case: { $eq: ["$courses.grade", "D+"] }, then: 1.3 },
|               { case: { $eq: ["$courses.grade", "D"] }, then: 1.0 },
|               { case: { $eq: ["$courses.grade", "F"] }, then: 0.0 }
|             ],
|             default: null
|           }
|         }
|       }
|     }
|   },
|   {
|     $project: {
|       _id: 0,
|       course_id: "$_id.course_id",
|       course_name: "$_id.course_name",
|       average_grade: { $round: ["$average_grade", 2] }
|     }
|   },
|   { $sort: { course_id: 1 } }
| ])
```

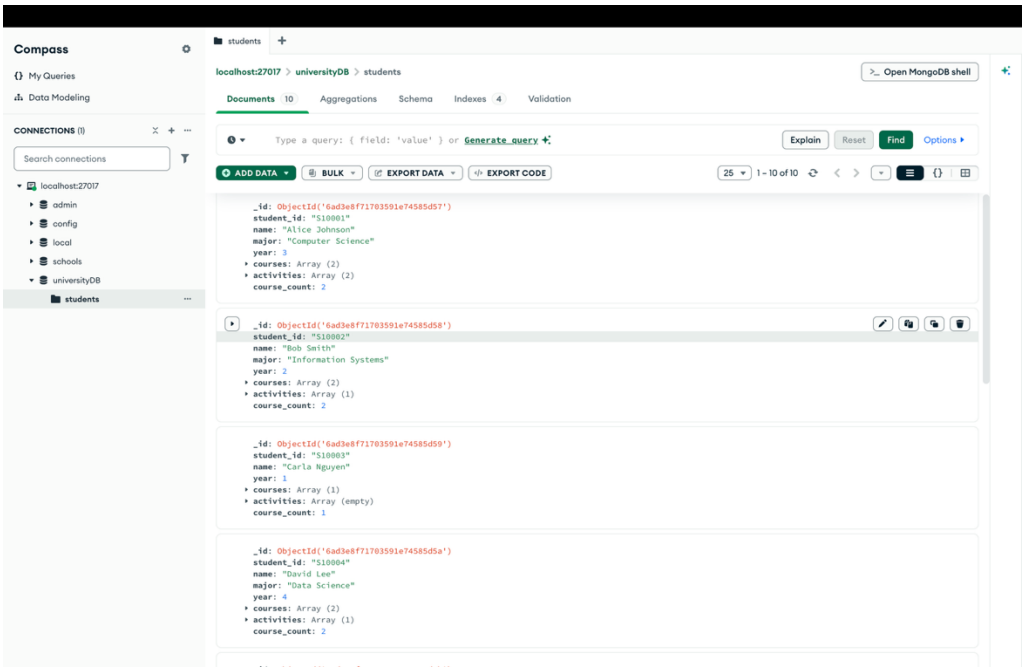
```
[
  {
    course_id: 'CS501',
    course_name: 'Advanced Databases',
    average_grade: 3.77
  },
  { course_id: 'CS502', course_name: 'Data Mining', average_grade: 3.5 },
  {
    course_id: 'CS503',
    course_name: 'Cloud Databases',
    average_grade: 3
  },
  {
    course_id: 'DS701',
    course_name: 'Machine Learning',
    average_grade: 3.65
  },
  {
    course_id: 'DS702',
    course_name: 'Big Data Systems',
    average_grade: 3.7
  },
  {
    course_id: 'IS601',
    course_name: 'Cloud Computing',
    average_grade: 3.7
  },
  { course_id: 'IS602', course_name: 'Cybersecurity', average_grade: 3 },
  {
    course_id: 'SE601',
    course_name: 'Software Architecture',
    average_grade: 3.3
  }
]
```

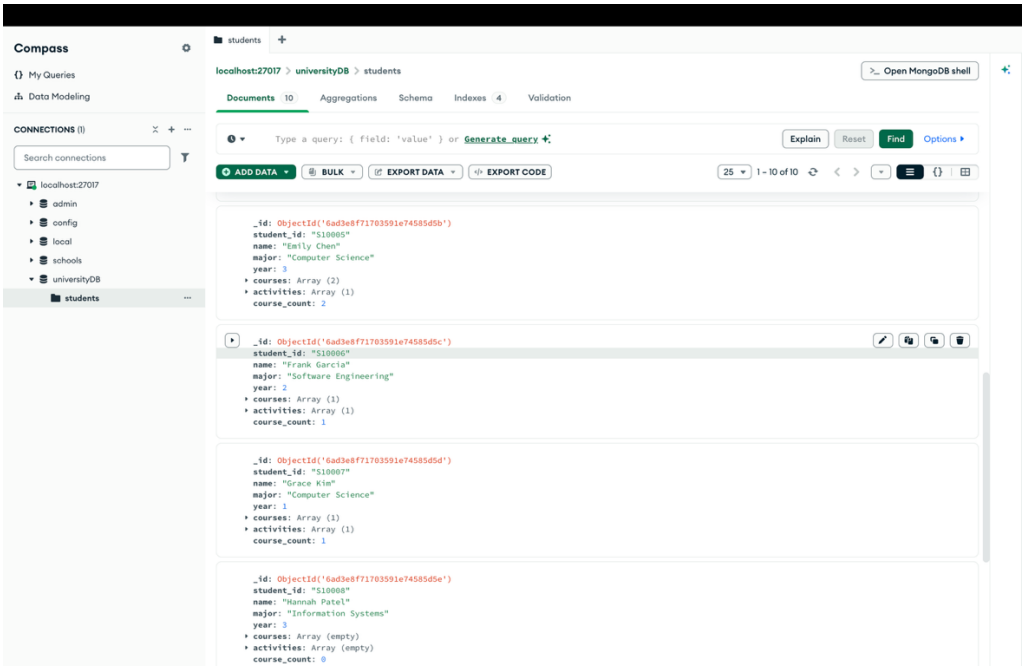
2. List majors and the number of students in each major.

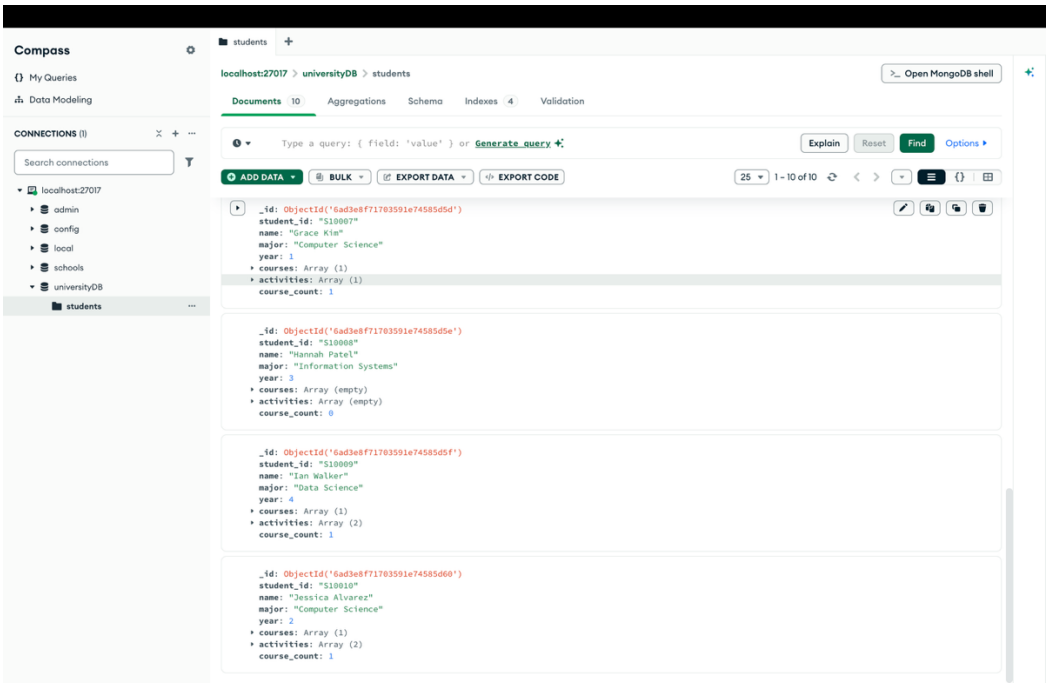
```
universityDB> db.students.aggregate([
|   {
|     $group: {
|       _id: "$major",
|       student_count: { $sum: 1 }
|     }
|   },
|   {
|     $project: {
|       _id: 0,
|       major: "$_id",
|       student_count: 1
|     }
|   },
|   {
|     $sort: { major: 1 }
|   }
| ])
[
  { student_count: 1, major: null },
  { student_count: 4, major: 'Computer Science' },
  { student_count: 2, major: 'Data Science' },
  { student_count: 2, major: 'Information Systems' },
  { student_count: 1, major: 'Software Engineering' }
]
universityDB>
```

3. For each student, add a new field "course_count" indicating how many courses they have taken.

```
universityDB> db.students.updateMany(
|   {},
|   [
|     {
|       $set: {
|         course_count: {
|           $size: { $ifNull: ["$courses", []] }
|         }
|       }
|     }
|   ]
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 10,
  modifiedCount: 10,
  upsertedCount: 0
}
universityDB>
```


Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
localhost:27017
admin
config
local
schools
universityDB
students
students
localhost:27017 > universityDB > students
Open MongoDB shell
Documents 10
Aggregations
Schema
Indexes 4
Validation
Type a query: { field: 'value' } or Generate query
Explain
Reset
Find
Options
ADD DATA
BULK
EXPORT DATA
EXPORT CODE
25
1 – 10 of 10
_id: ObjectId('6ad3e8f71703591e74585d57')
student_id: "S10001"
name: "Alice Johnson"
major: "Computer Science"
year: 3
courses: Array (2)
activities: Array (2)
course_count: 2
_id: ObjectId('6ad3e8f71703591e74585d58')
student_id: "S10002"
name: "Bob Smith"
major: "Information Systems"
year: 2
courses: Array (2)
activities: Array (1)
course_count: 2
_id: ObjectId('6ad3e8f71703591e74585d59')
student_id: "S10003"
name: "Carla Nguyen"
year: 1
courses: Array (1)
activities: Array (empty)
course_count: 1
_id: ObjectId('6ad3e8f71703591e74585d5a')
student_id: "S10004"
name: "David Lee"
major: "Data Science"
year: 4
courses: Array (2)
activities: Array (1)
course_count: 2


Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
localhost:27017
admin
config
local
schools
universityDB
students
students
localhost:27017 > universityDB > students
Open MongoDB shell
Documents 10
Aggregations
Schema
Indexes 4
Validation
Type a query: { field: 'value' } or Generate query
Explain
Reset
Find
Options
ADD DATA
BULK
EXPORT DATA
EXPORT CODE
25
1 – 10 of 10
_id: ObjectId('6ad3e8f71703591e74585d5b')
student_id: "S10005"
name: "Emily Chen"
major: "Computer Science"
year: 3
courses: Array (2)
activities: Array (1)
course_count: 2
_id: ObjectId('6ad3e8f71703591e74585d5c')
student_id: "S10006"
name: "Frank Garcia"
major: "Software Engineering"
year: 2
courses: Array (1)
activities: Array (1)
course_count: 1
_id: ObjectId('6ad3e8f71703591e74585d5d')
student_id: "S10007"
name: "Grace Kim"
major: "Computer Science"
year: 1
courses: Array (1)
activities: Array (1)
course_count: 1
_id: ObjectId('6ad3e8f71703591e74585d5e')
student_id: "S10008"
name: "Hannah Patel"
major: "Information Systems"
year: 3
courses: Array (empty)
activities: Array (empty)
course_count: 0



Part 5 – Reflection

Unlike relational databases, MongoDB offers schema flexibility in that it does not require a pre-defined schema before storing data. One can insert documents without confining to a strict constraint. Developers can capture data immediately and adapt the format of the document as requirement evolve as opposed to spending an immense time on creating a pre-defined schema. As seen in the student's collection, there was document that did not have a major field while other documents did. In addition, MongoDB prefers de-normalization as opposed to relational databases, allowing embedded documents and arrays. Single query can fetch the entire document instantly as opposed to relational databases, where JOIN operations must be performed across tables.

Using embedded documents and arrays changes the way one approaches querying in that it makes the querying much easier because relational joins are not needed. Querying with embedded documents and arrays become hierarchical traversals that are single operations. One of the most challenging parts of working with MongoDB is the query language. It was very

different from SQL and using MongoDB's query language felt a bit unnatural at first. I had moment where I had to look up how to perform a specific query and had issues getting used to the syntax as I would at times forget to enclose it in curly braces. In addition, despite the flexibility being considered a positive, managing the schema can be difficult in that there is no schema enforcer. Adding a new field to document does not mean that all documents existing are changed, which can cause nonuniformity across documents.